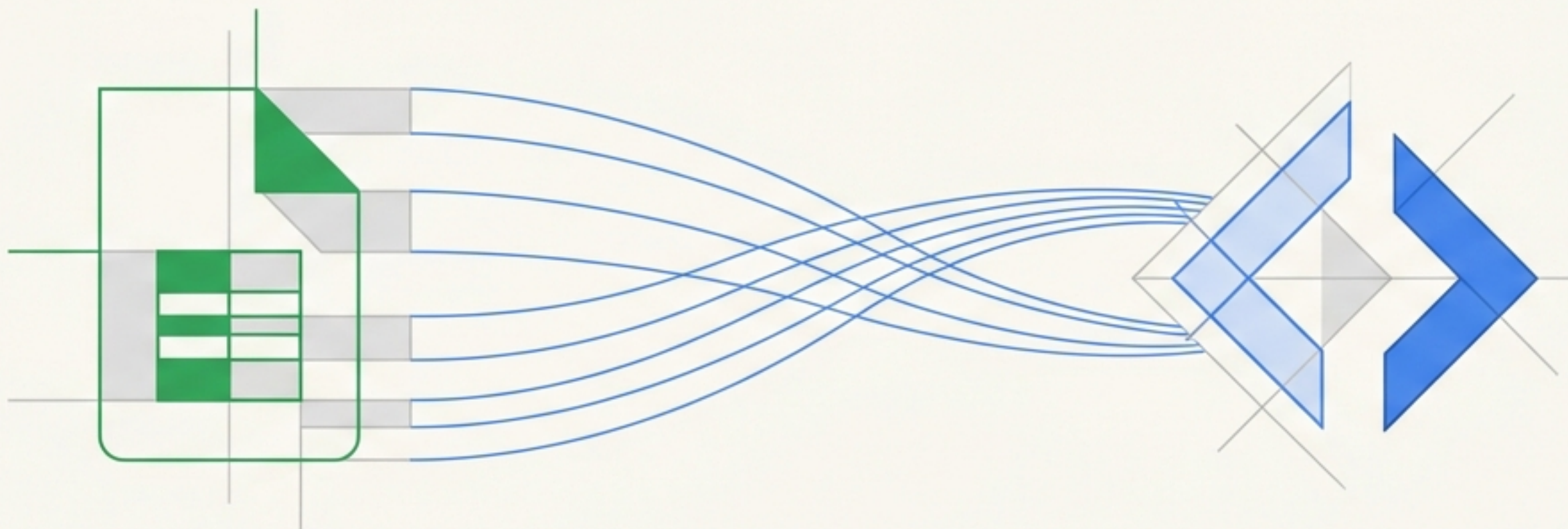


ชั่วโมงที่ 4: สร้าง Web API ด้วย Google Apps Script

จัดการข้อมูล (CRUD) ผ่าน Google Sheets แบบ Serverless



เป้าหมายการเรียนรู้ (Learning Objectives)

- ☐ เข้าใจแนวคิด Serverless และบทบาทของ Google Apps Script
- ☐ เตรียม Google Sheets เพื่อใช้เป็น 'Database' สำหรับโปรเจกต์
- ☐ เขียนโค้ดจัดการข้อมูลพื้นฐาน (Create และ Read)
- ☐ Deploy Script ให้กลายเป็น Web API ที่ใช้งานได้จริง
- ☐ ทดสอบเรียก API ผ่าน URL และเห็นผลลัพธ์เป็นข้อมูล JSON

Serverless: โฟกัสที่ Code ไม่ต้องกังวลเรื่อง Server



ไม่ต้องติดตั้ง Server เอง:
เราไม่ต้องจัดการกับฮาร์ดแวร์หรือระบบปฏิบัติการ



ไม่ต้องดูแลฐานข้อมูลหรือ Hosting: แพลตฟอร์มจะจัดการให้ทั้งหมด



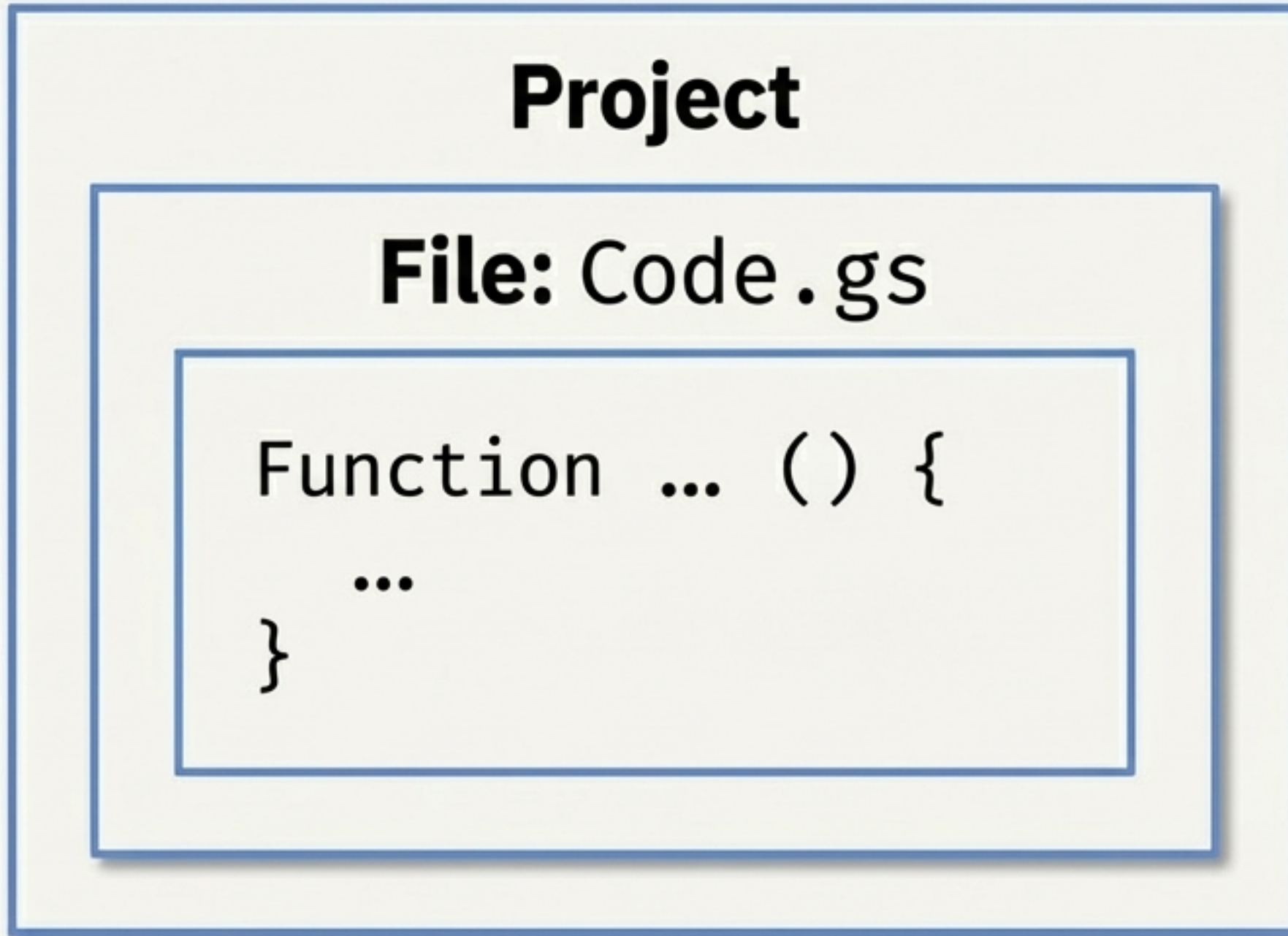
ใช้บริการ Cloud ที่มีให้แล้ว:
เราเพียงแค่นำโค้ดของเราไปวางและใช้งาน

เครื่องมือของเรา: Google Apps Script

- เป็น JavaScript ที่ทำงาน (รัน) อยู่บน Google Cloud
- สามารถเชื่อมต่อกับบริการต่างๆ ของ Google ได้โดยตรง (เช่น Sheets, Drive)
- ในโปรเจกต์นี้ เราจะใช้ Apps Script เป็น **Backend** สำหรับ Web Application ของเรา

**Google Sheets =
Database ของเรา**

โครงสร้างพื้นฐานของ Project



Project: คือพื้นที่ทำงานทั้งหมดของเรา

File (Code.gs): คือไฟล์ที่เราจะใช้เขียนโค้ด JavaScript

Function: คือชุดคำสั่งที่เราจะสร้างขึ้นเพื่อทำงานต่างๆ

`doGet(e)`: ประตูลู่ Web API ของเรา

- เป็นฟังก์ชัน ‘พิเศษ’ ที่จะถูกเรียกทำงานโดยอัตโนมัติเมื่อมีคนเรียกเข้ามาที่ URL ของเรา (ด้วยวิธี GET Request)
- หน้าหลักของมันคือ ‘ส่ง’ ข้อมูลออกไปให้กับผู้เรียก

```
function doGet(e) {  
    return ContentService.createTextOutput("Hello World");  
}
```


CRUD: หัวใจของการจัดการข้อมูล

Operation	ความหมาย	สิ่งที่เราจะลงมือทำ
Create	สร้าง / เพิ่มข้อมูลใหม่	<code>appendRow()</code>
Read	อ่าน / ดึงข้อมูล	<code>getValues()</code>
Update	แก้ไข / อัปเดตข้อมูล	<code>setValue()</code>
Delete	ลบข้อมูล	<code>deleteRow()</code>

ลงมือเขียน Code: (R) Read - การอ่านข้อมูลจาก Sheet

เราจะแก้ไขฟังก์ชัน `doGet()` เพื่อให้ดึงข้อมูลทั้งหมดจาก Sheet ที่เราเตรียมไว้

```
function doGet() {  
  let sheet = SpreadsheetApp  
    .getActiveSheet()  
    .getSheetByName("data");  
  
  let data = sheet.getDataRange().getValues();  
  
  return ContentService  
    .createTextOutput(JSON.stringify(data))  
    .setMimeType(ContentService.MimeType.JSON);  
}
```


ทำความเข้าใจ Code `doGet()` ที่ละส่วน

```
function doGet() {  
  let sheet = SpreadsheetApp  
    .getActiveSheet()  
    .getSheetByName("data");  
  
  let data = sheet.getDataRange().getValues();  
  
  return ContentService  
    .createTextOutput(JSON.stringify(data))  
    .setMimeType(ContentService.MimeType.JSON);  
}
```

- เชื่อมต่อไปยัง Sheet ที่ชื่อ 'data'
- ดึงข้อมูลทั้งหมดใน Sheet ออกมาในรูปแบบ Array 2 มิติ
- แปลงข้อมูล Array ให้กลายเป็น JSON String เพื่อส่งผ่านเว็บ
- คือการบอก Browser ว่าข้อมูลที่ส่งกลับมานี้เป็นชนิด JSON นะ

ลงมือเขียน Code: (C) Create – การเพิ่มข้อมูลใหม่

เราจะใช้ฟังก์ชัน `doPost(e)` สำหรับ 'รับ' ข้อมูลที่ถูกส่งเข้ามา (ด้วยวิธี POST Request) เพื่อนำมาบันทึกลง Sheet

```
function doPost(e) {  
  let sheet = SpreadsheetApp  
    .getActiveSheet()  
    .getSheetByName("data");  
  
  let obj = JSON.parse(e.postData.contents);  
  
  sheet.appendRow([  
    obj.id,  
    obj.name,  
    obj.score  
  ]);  
  
  return ContentService.createTextOutput("Insert success");  
}
```


แนวคิดเบื้องต้น: (U) Update – การแก้ไขข้อมูล

- **หลักการทำงาน**:** เราต้องวน Loop เพื่อหาแถวที่มี `id` ตรงกับที่ต้องการ จากนั้นจึงใช้คำสั่ง `setValue()` เพื่อแก้ไขข้อมูลในช่องนั้นๆ
หมายเหตุ: ในชั่วโมงนี้ เราจะดูโค้ดเพื่อทำความเข้าใจหลักการทำงานก่อน

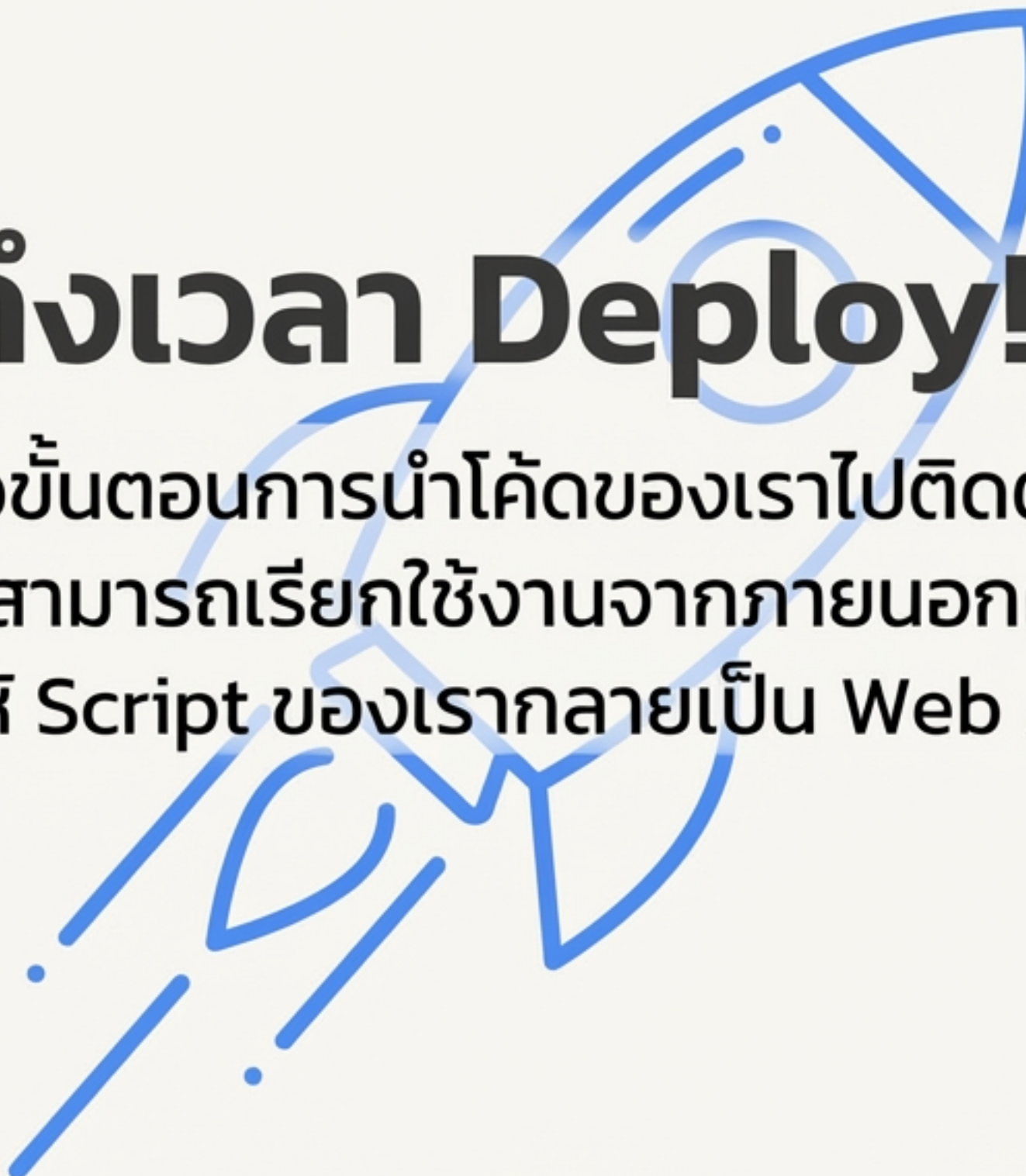
```
function updateData(id, newScore) {  
  let sheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName("data");  
  let data = sheet.getDataRange().getValues();  
  
  for (let i = 1; i < data.length; i++) {  
    if (data[i][0] == id) { // [0] คือคอลัมน์ id  
      sheet.getRange(i + 1, 3).setValue(newScore); // 3 คือคอลัมน์ score  
    }  
  }  
}
```

คอลัมน์ id จะอยู่ใน index 0 และคอลัมน์ score อยู่ใน index 2 (3-1) ของแต่ละแถว

แนวคิดเบื้องต้น: (D) Delete – การลบข้อมูล

- **หลักการทำงาน**:** คล้ายกับการ Update, เราจะวน Loop เพื่อหาแถวที่มี `id` ที่ต้องการ จากนั้นใช้คำสั่ง `deleteRow()` เพื่อลบทั้งแถว
หมายเหตุ: การวน Loop จากท้ายขึ้นมาจะปลอดภัยกว่าเมื่อมีการลบแถว เพื่อไม่ให้ลำดับแถวเพี้ยน

```
function deleteData(id) {  
  let sheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName("data");  
  let data = sheet.getDataRange().getValues();  
  
  for (let i = data.length - 1; i > 0; i--) {  
    if (data[i][0] == id) {  
      sheet.deleteRow(i + 1);  
    }  
  }  
}
```

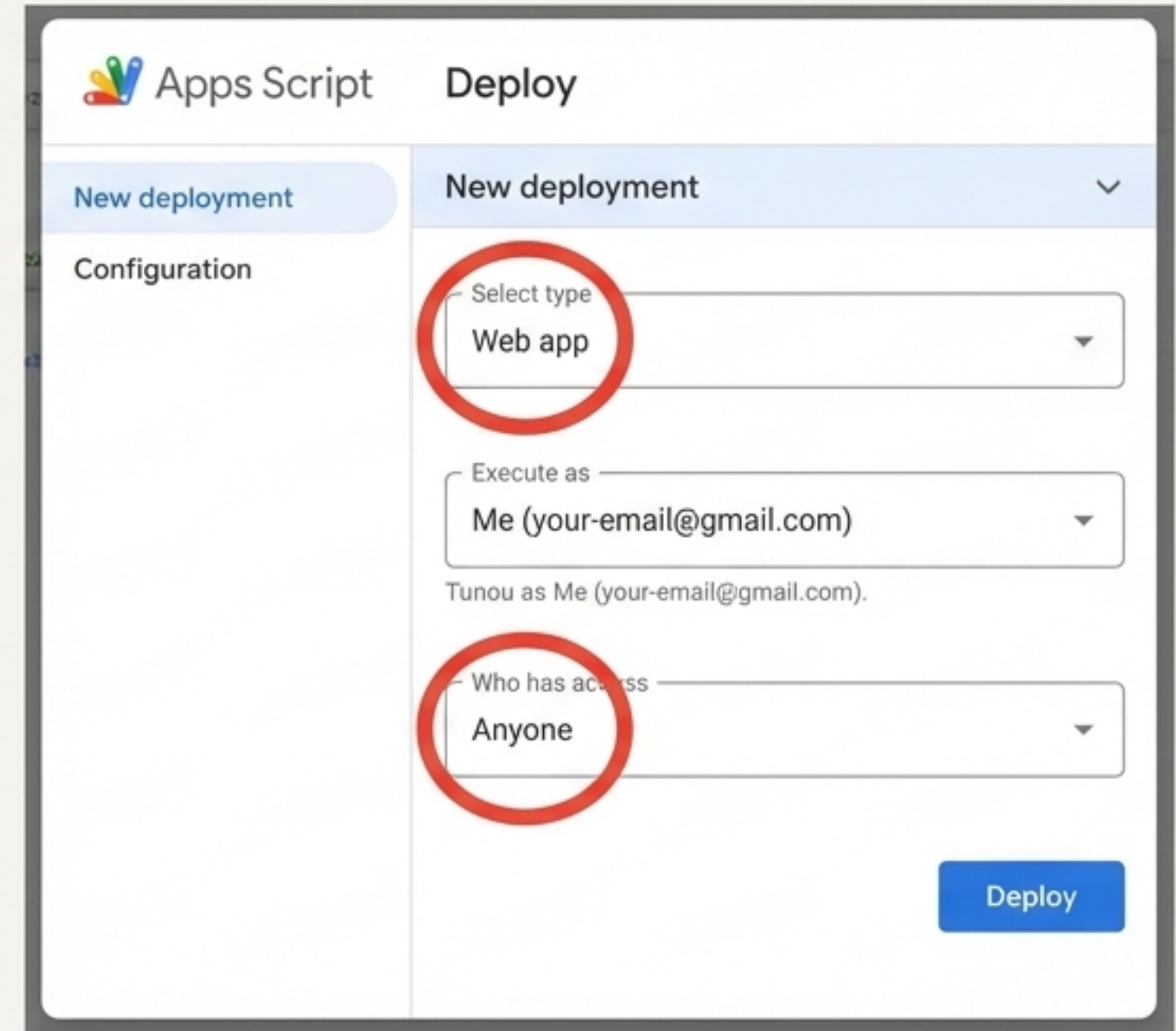



ถึงเวลา Deploy!

การ Deploy คือขั้นตอนการนำโค้ดของเราไปติดตั้งบน Google Cloud เพื่อให้สามารถเรียกใช้งานจากภายนอกผ่าน URL ได้ มันคือการทำให้ Script ของเรากลายเป็น Web App ที่มีชีวิต

ลงมือทำ: 5 ขั้นตอนสู่ Web API

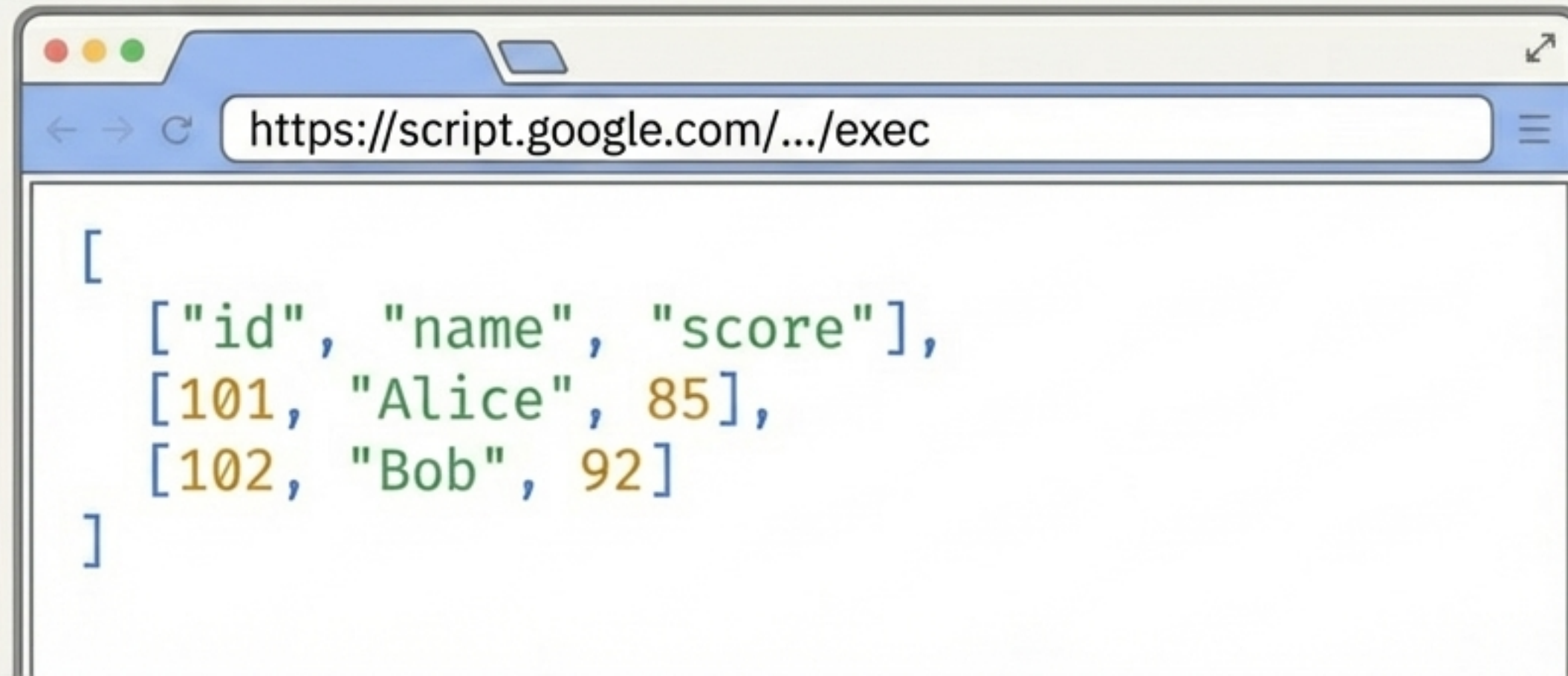
1. คลิกปุ่ม **Deploy** (สีฟ้ามุมบนขวา) → เลือก **New deployment.**
2. คลิกไอคอนรูปเฟือง ⚙️ และเลือกประเภทเป็น **Web app.**
3. ในช่อง 'Execute as': เลือก **Me (your-email@gmail.com).**
4. ในช่อง 'Who has access': เลือก **Anyone.**
5. คลิก **Deploy**, กด **Authorize access** เพื่อให้สิทธิ์, และสุดท้าย **คัดลอก** (Copy) **Web app URL** ที่ได้มา.



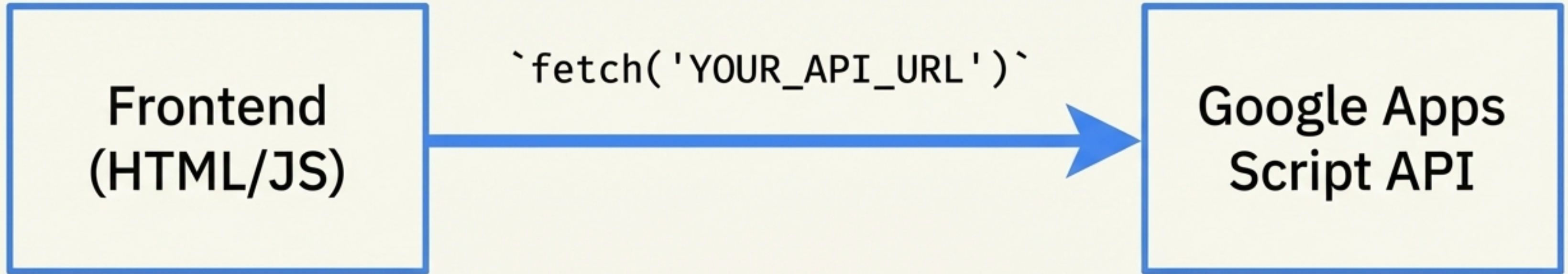
กิจกรรมที่ 2: ทดสอบ API ที่มีชีวิตของเรา!

นำ URL ที่คัดลอกไว้จากขั้นตอนที่แล้ว ไปวางในแถบที่อยู่ของ Web Browser แล้วกด Enter

ผลลัพธ์ที่คาดหวัง: คุณจะเห็นข้อมูลจาก Google Sheet ของคุณแสดงผลเป็นรูปแบบ JSON บนหน้าจอ!



แล้วเราจะนำ URL นี้ไปใช้อย่างไรต่อ?



- URL ที่เราได้มานี้ คือ Serverless API ของเราอย่างสมบูรณ์
- ในชั่วโมงที่ 5 เราจะเรียนรู้การใช้คำสั่ง `fetch()` ใน JavaScript ฟัง Frontend เพื่อเรียกใช้งาน API นี้
- เป้าหมายคือสร้าง Web Application ที่ทำงานได้ครบวงจร (Client ↔ Server)

บททวน: คำสั่งหลักในการจัดการข้อมูล

Operation	ความหมาย	Apps Script Method
Create	เพิ่มข้อมูล	<code>sheet.appendRow([...])</code>
Read	อ่านข้อมูล	<code>sheet.getDataRange().getValues()</code>
Update	แก้ไขข้อมูล	<code>range.setValue(...)</code>
Delete	ลบข้อมูล	<code>sheet.deleteRow(...)</code>

สิ่งที่คุณทำสำเร็จในวันนี้ (Your Achievements)

- ✓ อธิบายแนวคิด Serverless และใช้ Google Apps Script เป็น Backend ได้
- ✓ ใช้ Google Sheets เป็นฐานข้อมูลสำหรับเก็บข้อมูล
- ✓ เขียนโค้ดเพื่ออ่านและเพิ่มข้อมูล (CRUD) ได้
- ✓ Deploy Script เป็น Web App ที่ใช้งานได้จริง
- ✓ เรียกใช้งาน API ที่สร้างขึ้นเองผ่าน URL และเห็นผลลัพธ์!

ถาม-ตอบ (Q&A)

ชั่วโมงถัดไป: ชั่วโมงที่ 5 – เชื่อมต่อ Frontend ด้วย fetch()
เพื่อสร้าง Web Application ที่สมบูรณ์

